INT1405 - DISTRIBUTED SYSTEMS

LECTURE 09

Security in Distributed Systems Concepts & Design

Semester 1, 2026-2027

Lê Hà Thanh
lehathanh@ptithcm.edu.vn

Tanenbaum & van Steen (2nd Edition), Chapter 9 (Section 9.1 & 9.2)

Content

Module 1: Security Threats, Policies, & Architectural Design Principles.

Module 2: Cryptographic Foundations (Symmetric, Asymmetric, Hash Functions).

Module 3: Secure Channels, Authentication Protocols, & Kerberos Architecture.

Module 4: Access Control, Capabilities, Firewalls, & Mobile Code Security.

Module 5: Academic Assessment, Numerical Problems, & Practical Lab.

SLIDE 02

Learning Objectives & Pedagogical Roadmap

Key Analytical Competencies

Differentiate Security Threats: Interception, Interruption, Modification, Fabrication.

Contrast Policy vs. Mechanism: Define formal security requirements vs. technical implementations.

Evaluate Architectural Layers: Analyze security placement in Middleware, OS, and Network layers.

Apply TCB & RISSC: Minimize the Trusted Computing Base using component isolation.

Practical Technical Skills

Analyze Cryptographic Algorithms: Compare AES, RSA, SHA-256, and HMAC primitives.

Evaluate Authentication Protocols: Trace Needham-Schroeder, Kerberos v5, and TLS 1.3 handshakes.

Master Access Control Models: Implement ACLs, Capabilities, and Role-Based Access Control (RBAC).

Solve Security Calculations: Compute RSA modular arithmetic, Diffie-Hellman keys, and threshold signatures.

SLIDE 03

Security vs. Dependability: The CIA Triad

Dependability & Security Intersection

Dependability Scope: Availability, Reliability, Safety, Maintainability, and Security.

Trust vs. Technical Security: Security is technical/provable; Trust is emotional/assumed.

System Assets: Protecting Hardware, Software, Data, and System Services from corruption.

The CIA Triad Guarantees

Confidentiality: Information is disclosed only to authorized entities;

- prevents illegal eavesdropping.

Integrity: Alterations to system assets can be made only in an authorized, detectable manner.

Availability: Services and data remain accessible to authorized users despite attacks or failures.

SLIDE 04

Taxonomy of Security Threats: The 4 Threat Classes

Passive Security Threats

Interception: An unauthorized party gains access to a service or private data.

Examples: Network packet sniffing, eavesdropping on RPC channels, illegally copying files.

Characteristics: Hard to detect immediately; primarily attacks Confidentiality.

Active Security Threats

Interruption: A service becomes unavailable or unusable (attacks Availability; e.g., DDoS).

Modification: An unauthorized party tampers with transmitted data (attacks Integrity).

Fabrication: An intruder inserts fake messages or false identities into the network (attacks Authentication).

SLIDE 05

Security Policies vs. Security Mechanisms

Security Policy (What is Allowed)

Definition: A precise statement of rules defining which actions subjects are permitted to perform on resources.

Specification: Defines

- access rights,
- user roles,
- security domains, and
- data sensitivity levels.

Independence: Formulated independently of technical implementation details.

Security Mechanism (How to Enforce)

Definition: Technical protocols, software primitives, and hardware enforcement mechanisms.

Functions:

- Encryption,
- Digital Signatures,
- Authentication Handshakes,
- Access Control Lists.

Separation Principle: Policy MUST be separated from mechanism to allow flexible policy changes without re-engineering.

SLIDE 06

Case Study: Globus Grid Security Policy Architecture

Globus Security Context

Environment: Wide-area grid computing with heterogeneous, cross-administrative organizational domains.

Requirement: Single sign-on across multi-institutional compute clusters without sharing root credentials.

Globus Security Principles

User Mapping: Maps global grid credentials (X.509) to local OS user accounts dynamically.

Proxy Certificates: Issues short-lived delegated credentials for sub-job submission across sites.

Policy Enforcement: Local site security policies retain ultimate authority over resource access.

SLIDE 07

Architectural Design Issues: Focus of Control

Data-Centric Focus of Control

Focus: Protecting individual data objects directly against invalid operations.

Mechanism: Object-level encapsulation, methods, and cryptographic object signatures.

Invocation & User Focus of Control

Invocation Control: Protecting operations against unauthorized caller invocations (Method-level ACLs).

User Control: Protecting resources by verifying the identity and role of the initiating user.

Trade-off: Finer granularity improves protection but increases authorization overhead.

SLIDE 08

Layering Security Mechanisms & Network Protocol Stack

Low-Level Security Placement

Network / Link Layer (IPsec / SMDS): Encrypts all packets between physical routers/switches.

Advantage: Transparent to applications; secures all traffic automatically.

Disadvantage: Does not protect against internal host attacks or application-level fraud.

High-Level Security Placement

Transport / Middleware Layer (TLS / SSL / Secure RPC): Secures end-to-end communication channels.

Advantage: End-to-end security; provides fine-grained user and application authentication.

Trust Dependencies: Middleware security relies strictly on the security of lower OS kernel layers.

SLIDE 09

The Trusted Computing Base (TCB) & Minimization

Trusted Computing Base (TCB) Definition

Scope: The set of all hardware, software, and procedural mechanisms required to enforce a security policy.

Trust Requirement: The entire system's security fails if any single component inside the TCB is compromised.

Components:

- OS Kernel,
- Authentication Daemons,
- Key Management Systems,
- Hardware Cryptoprocessors.

TCB Minimization Principle

Design Goal: Keep the TCB as small and simple as possible to enable formal verification and auditing.

Vulnerability Rule: TCB complexity is proportional to security vulnerabilities (more lines of code = more bugs).

Isolation Strategy: Move non-critical services outside the TCB into user-space applications.

SLIDE 10

RISSC Architecture: Reduced Interfaces for Secure Components

RISSC Architectural Philosophy

Reduced Interfaces for Secure System Components (RISSC): Isolates security-critical servers on dedicated hardware.

Hardware Separation: Clients run on untrusted commodity machines; secure databases run on locked-down servers.

Secure Network Interface Filtering

Access Control Devices: Hardware filtering devices block direct network access to secure servers.

Restricted Interface: Clients interact with secure servers strictly via audited, narrow RPC APIs.

Impact: Prevents untrusted client OS compromises from directly corrupting core database state.

Content

Module 1: Security Threats, Policies, & Architectural Design Principles.

Module 2: Cryptographic Foundations (Symmetric, Asymmetric, Hash Functions).

Module 3: Secure Channels, Authentication Protocols, & Kerberos Architecture.

Module 4: Access Control, Capabilities, Firewalls, & Mobile Code Security.

Module 5: Academic Assessment, Numerical Problems, & Practical Lab.

SLIDE 11

Introduction to Cryptographic Foundations

Role of Cryptography in Distributed Systems

Fundamental Tool: Cryptography transforms readable plaintext into unintelligible ciphertext.

Security Objectives:

- Guarantees Confidentiality,
- Message Integrity,
- Authentication,
- Non-Repudiation.

Kerckhoffs's Principle: Security must depend ONLY on key secrecy, not on keeping algorithm design secret.

Cryptographic Operations & Notation

Encryption Function:

$$C = E_K(P)$$

- Plaintext P encrypted with key K yields Ciphertext C.

Decryption Function:

$$P = D_K(C)$$

- Ciphertext C decrypted with key K yields Plaintext P.

Key Space: Mathematical space of all valid key values;

- must be large enough to resist brute-force search.

SLIDE 12

Symmetric (Secret-Key) vs. Asymmetric (Public-Key) Ciphers

Symmetric Key Cryptosystems

Shared Secret: Sender and Receiver use the EXACT same secret key K_{AB} for encryption and decryption.

Performance: Extremely fast computation (gigabytes/sec); hardware accelerated.

Key Scalability Problem: Requires $N(N-1)/2$ keys for N processes to communicate pairwise.

Asymmetric Key Cryptosystems

Key Pair: Entity holds Public Key K_{pub} (distributed freely) and Private Key K_{priv} (kept secret).

Mathematical Property:

$$C = E_{K_{pub}}(P) \rightarrow P = D_{K_{priv}}(C)$$

Key Scalability: Requires only $2N$ keys for N processes;

- slower execution than symmetric ciphers.

SLIDE 13

Symmetric Block Ciphers: DES, AES & Modes of Operation

Symmetric Algorithms (AES & DES)

Data Encryption Standard (DES): 56-bit key length; obsolete due to small key space (susceptible to brute force).

Advanced Encryption Standard (AES): Standard substitution-permutation cipher with 128, 192, or 256-bit keys.

Block Processing: Processes plaintext in fixed 128-bit blocks.

Block Cipher Modes: ECB vs. CBC

Electronic Codebook (ECB): Identical plaintext blocks produce identical ciphertext blocks (INSECURE!).

Cipher Block Chaining (CBC): XORs plaintext block with previous ciphertext block using an Initialization Vector (IV).

Property: Ensures identical plaintext blocks yield completely different ciphertext outputs.

SLIDE 14

Asymmetric Ciphers: RSA Mathematical Foundations

RSA Algorithm Construction (Rivest, Shamir, Adleman)

1. **Prime Selection**: Pick two large prime numbers p and q; compute modulus $n = p * q$.

2. **Euler Totient**: Compute

$$phi(n) = (p - 1) * (q - 1)$$

3. **Public Exponent**: Choose e such that

$$\gcd\big(e, phi(n)\big) = 1$$

4. **Private Exponent**: Calculate d such that

$$(d * e)\ mod\ phi(n) = 1$$

RSA Encryption & Decryption Mechanics

Public Key: (e, n) | **Private Key**: (d, n).

Encryption: Ciphertext

$$C = (P^e)\ mod\ n$$

Decryption: Plaintext

$$P = \big(C^d\big)\ mod\ n$$

Hardness Assumption: Factoring large composite integer n into p and q is computationally intractable.

SLIDE 15

Hybrid Cryptosystems: Performance Optimization

The Performance Dilemma

Symmetric Performance: High throughput

- (AES = ~1-10 Gbps per core).

Asymmetric Performance: Low throughput

- (RSA-2048 = ~100-1000 ops/sec; 1000x slower than AES).

Problem: Encrypting megabyte/gigabyte payloads directly with RSA is computationally prohibitive.

Hybrid Cryptosystem Execution

Step 1: Sender generates a fast, random symmetric Session Key K_{sess} (e.g., a 256-bit AES key).

Step 2: Sender encrypts large message data payload with

$$K_{sess}\colon C_{data} = E_{K_{sess}}(Message)$$

Step 3: Sender encrypts K_{sess} with receiver's RSA Public Key:

$$C_{key} = E_{K_{pub}}(K_{sess})$$

Step 4: Receiver decrypts C_{key} using RSA Private Key, then decrypts C_{data} using K_{sess}.

SLIDE 16

Cryptographic Hash Functions & Message Digests

Hash Function Requirements

One-Way Function (Pre-image Resistance): Given hash $H(m)$, it is computationally impossible to find m.

Second Pre-image Resistance: Given message m_1, impossible to find m_2 such that

$$H(m_1) = H(m_2)$$

Collision Resistance: **Impossible** to find ANY two distinct messages m_1, m_2 such that

$$H(m_1) = H(m_2)$$

Common Hash Algorithms: MD5 vs. SHA-256

MD5 (128-bit digest): Obsolete; collision attacks execute in seconds (DO NOT USE).

SHA-1 (160-bit digest): Weakened; deprecated by security standards.

SHA-256 / SHA-3 (256-bit digest): Modern secure hash standards; widely used for digital signatures.

SLIDE 17

Digital Signatures & Non-Repudiation

Digital Signature Properties

Authenticity: Proves message originated from the specific claimed sender.

Integrity: Proves message was not modified in transit.

Non-Repudiation: Sender cannot deny having sent the message once signed.

Digital Signature Operation Sequence

1. Signature Generation: Sender computes hash digest $d = H(m)$, then encrypts d with Private Key: $Sig = E_{K_{priv}}(d)$.

2. Signature Transmission: Sender transmits `(Message m, Signature Sig).`

3. Signature Verification: Receiver computes $d_1 = H(m)$ and decrypts Sig with Sender's Public Key: $d_2 = D_{K_{pub}}(Sig)$.

4. Check: If $d_1 == d_2$, signature is valid!

SLIDE 18

Message Authentication Codes (HMAC)

Keyed Hash Functions (HMAC)

Purpose: Provides message integrity and sender authentication using shared secret keys.

Mechanism: Combines a secret key K_{AB} with message m inside a cryptographic hash function.

Formula:

$$HMAC(K, m) = H\left((K\ XOR\ opad) \parallel H\big((K\ XOR\ ipad) \parallel m\big)\right)$$

HMAC vs. Digital Signatures

Symmetric Performance: HMAC computation is extremely fast compared to asymmetric digital signatures.

No Non-Repudiation: Because K_{AB} is shared between Alice and Bob, either could have generated the HMAC.

Use Case: Protecting high-speed network packet integrity in IPsec and TLS data channels.

SLIDE 19

Diffie-Hellman Key Exchange Protocol

Diffie-Hellman Mathematical Concept

Goal: Allows two parties to establish a shared secret key over an INSECURE channel without pre-shared keys.

Mathematical Hardness: Based on the difficulty of computing Discrete Logarithms in finite prime fields.

Diffie-Hellman Protocol Exchange

1. **Setup**: Agree on public large prime p and generator g.

2. **Secret Keys**: Alice picks private key a; Bob picks private key b.

3. **Public Exchange**:
   - Alice sends $A = (g^a) \bmod p$;
   - Bob sends $B = (g^b) \bmod p$.

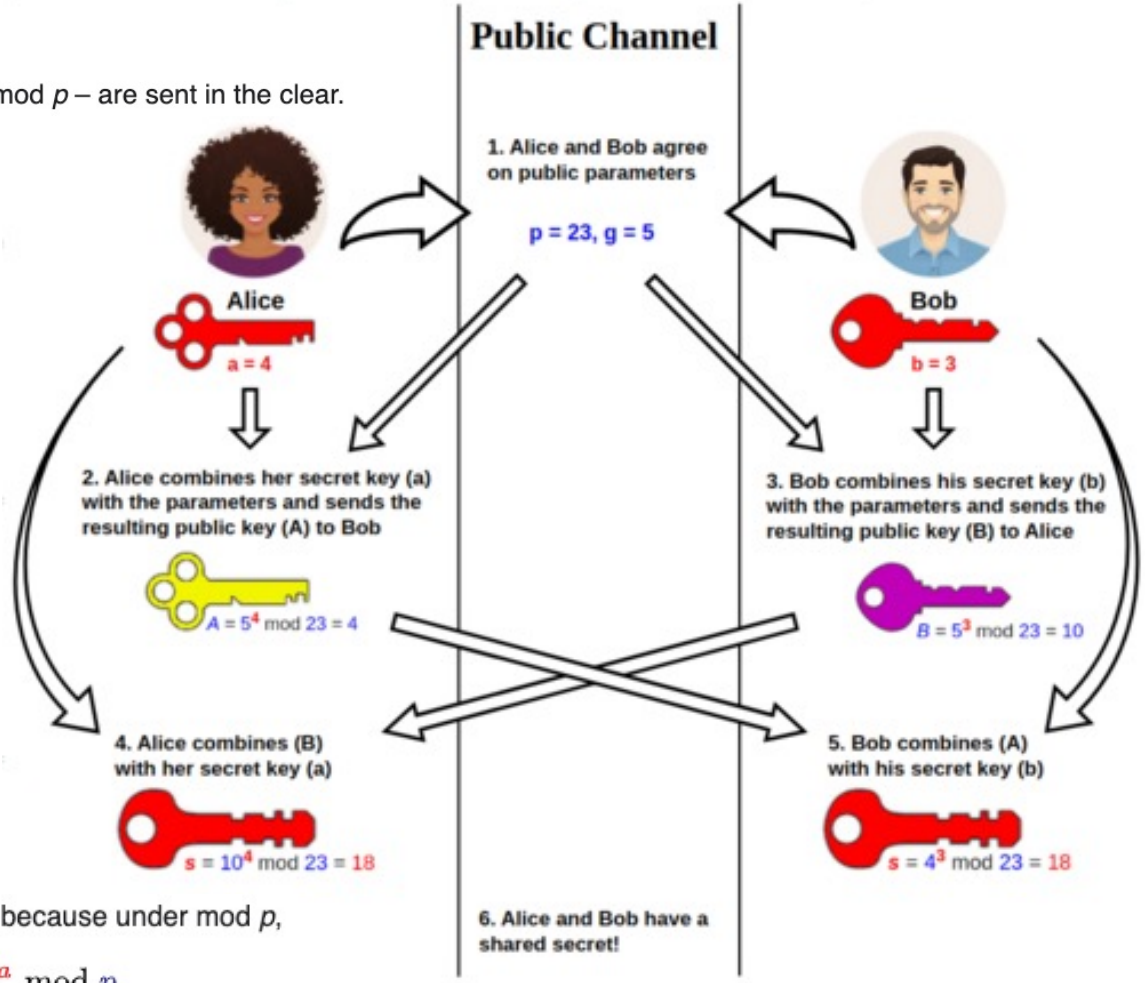
4. **Shared Key**: Each person takes the other's public value and raises it to their own private key:
   - Alice computes $K = (B^a) \bmod p$;
   - Bob computes $K = (A^b) \bmod p$.
   - Both get $K = (g^{ab}) \bmod p$!

SLIDE 19

Diffie-Hellman Key Exchange Protocol

Only *a* and *b* are kept secret.

All the other values – *p*, *g*, g^a mod *p*, and g^b mod *p* – are sent in the clear.



Both Alice and Bob have arrived at the same values because under mod *p*,

$$A^b \bmod p = g^{ab} \bmod p = g^{ba} \bmod p = B^a \bmod p$$

More specifically,

$$(g^a \bmod p)^b \bmod p = (g^b \bmod p)^a \bmod p$$

Source: https://en.wikipedia.org/wiki/Diffie%E2%80%93Hellman_key_exchange#/media/File:DiffieHellman.png

SLIDE 20

Diffie-Hellman Man-in-the-Middle (MitM) Attack

Vulnerability to Unauthenticated Exchanges

Flaw: Standard Diffie-Hellman does NOT authenticate the identities of exchanging parties.

Scenario: Intruder Eve intercepts messages between Alice and Bob over the network wire.

Man-in-the-Middle Execution Steps

1. Alice sends $A = (g^a)\ mod\ p$ to Bob.
    - Eve **intercepts** A and sends $E_1 = (g^{e_1})\ mod\ p$ to Bob.
2. Bob sends $B = \left(g^b\right)\ mod\ p$ to Alice.
    - Eve **intercepts** B and sends $E_2 = (g^{e_2})\ mod\ p$ to Alice.
3. *Result*:
    - Alice shares key K_{AE} with Eve;
    - Bob shares key K_{EB} with Eve.
    - Eve decrypts and re-encrypts all traffic!

Mitigation: Station-to-Station (STS) protocol; sign DH public parameters with RSA digital signatures.

SLIDE 21

Public Key Infrastructure (PKI) & Digital Certificates

The Public Key Distribution Problem

Problem: How does Bob know that public key $K_{pub_{Alice}}$ genuinely belongs to Alice and not to intruder Eve?

Solution: Digital Certificates issued by a mutually trusted Certification Authority (CA).

X.509 Digital Certificate Structure

Contents:

- Subject Name,
- Subject Public Key,
- Issuer CA Name,
- Validity Period,
- Serial Number.

CA Binding Signature:

$$Sig_{CA} = E_{K_{priv_{CA}}}\big(H(Subject\ Name \parallel Subject\ Public\ Key)\big)$$

Verification: Client verifies Sig_{CA} using the CA's well-known root public key.

SLIDE 22

Certificate Lifetimes, Revocation & CRLs / OCSP

Certificate Longevity & Revocation

Expiration: Certificates contain explicit Start Time and Expiration Time bounds.

Compromise Risk: If a subject's private key is leaked before expiration, the certificate MUST be revoked immediately.

Revocation Enforcement Mechanisms

1. **Certificate Revocation Lists (CRLs)**: CA periodically publishes a signed list of revoked certificate serial numbers.

2. **Online Certificate Status Protocol (OCSP)**: Client queries CA server in real-time to check certificate validity status.

OCSP Stapling: Web server fetches signed OCSP response and attaches it to TLS handshake to save client DNS lookups.

SLIDE 23

Trust Models: Hierarchical PKI vs. Web of Trust (PGP)

Hierarchical PKI Trust Model (X.509)

Structure: Tree hierarchy with Root CAs at the top, Intermediate CAs, and End-Entity Certificates.

Root Anchors: Root CA certificates pre-installed in OS and browser trust stores.

Limitation: Single Point of Failure; compromised Root CA can issue rogue certificates for any domain.

Web of Trust Model (PGP / GPG)

Structure: Decentralized peer-to-peer web; no central authority.

Cross-Signing: Users sign each other's public keys after verifying identity in person.

Trust Calculation: Client evaluates key validity based on the number of trusted peer signatures.

SLIDE 24

Cryptographic Nonces & Replay Attack Mitigation

Replay Attack Definition

Threat: Intruder intercepts valid encrypted authentication packet and re-transmits it later to gain unauthorized access.

Vulnerability: System accepts duplicate valid authentication packets if no freshness check exists.

Mitigation Mechanisms: Nonces vs. Timestamps

1. **Nonce (Number Used Once)**: Sender includes fresh random integer R; receiver verifies R has never been used before.

2. **Timestamps**: Sender includes current time T; receiver checks if

$$|T_{local} - T| < \delta_t$$

Trade-off:

- Timestamps require synchronized physical clocks;
- Nonces require storing previously seen random values.

Content

Module 1: Security Threats, Policies, & Architectural Design Principles.

Module 2: Cryptographic Foundations (Symmetric, Asymmetric, Hash Functions).

Module 3: Secure Channels, Authentication Protocols, & Kerberos Architecture.

Module 4: Access Control, Capabilities, Firewalls, & Mobile Code Security.

Module 5: Academic Assessment, Numerical Problems, & Practical Lab.

SLIDE 25

Secure Channels: Core Architecture & Properties

Secure Channel Definition

Concept: An abstraction establishing a protected communication pipe between client and server across untrusted networks.

Core Security Guarantees

1. **Mutual Authentication**: Confirms identity of both client and server before exchanging data.

2. **Message Confidentiality**: Encrypts payload traffic to prevent eavesdropping.

3. **Message Integrity & Freshness**: Uses MACs and sequence numbers to block tampering and replay attacks.

SLIDE 26

Authentication Protocols: Shared Secret Handshakes

Shared Key Authentication Mechanics

Premise: Alice and Bob share a secret symmetric key K_{AB} in advance.

Challenge-Response Protocol Sequence

1. **Alice -> Bob**: 'I am Alice'.

2. **Bob -> Alice**: Challenge Nonce R_B.

.3**Alice -> Bob**: Response $E_{K_{AB}}(R_B)$.

4. **Bob verifies**: Decrypts response with K_{AB}.

   If result matches R_B, Alice's identity is authenticated!

Mutual Authentication: Repeat challenge in reverse direction (Bob proves identity to Alice using R_A).

SLIDE 27

Authentication via Key Distribution Centers (KDCs)

The Key Management Scalability Problem

Pairwise Key Scaling: N hosts require $N(N-1)/2$ shared secret keys for pairwise security.

Scaling Limit: Managing thousands of pairwise keys on every host is administrative chaos.

KDC Centralized Key Distribution Architecture

Centralized KDC: Single trusted KDC shares secret key K_i with each host i (N keys total!).

Session Key Distribution: KDC generates temporary symmetric session key K_{AB} dynamically when Alice and Bob want to talk.

SLIDE 28

Needham-Schroeder Symmetric Key Protocol Sequence

Needham-Schroeder Protocol Message Flow

1. **Alice -> KDC**: $(Alice, Bob, R_A)$ [Request session key for Bob with Nonce R_A].

2. **KDC -> Alice**: $E_{K_A}(R_A, Bob, K_{AB}, Ticket_{Bob})$ where $Ticket_{Bob} = E_{(K_B)}(K_{AB}, Alice)$.

3. **Alice -> Bob**: Ticket_Bob, $E_{K_{AB}}(R_A)$ [Alice forwards ticket to Bob].

4. **Bob -> Alice**: $E_{K_{AB}}(R_B)$ [Bob decrypts ticket to get K_{AB}, sends challenge R_B].

5. **Alice -> Bob**: $E_{K_{AB}}(R_B - 1)$ [Alice responds to Bob's challenge].

Protocol Flaw & Denning-Sacco Fix

Replay Flaw: If old session key K_{AB} is compromised, intruder can replay old Ticket_Bob to impersonate Alice!

Denning-Sacco Fix: Add fresh timestamp to ticket or require Bob to supply a nonce.

SLIDE 29

Kerberos v5 Authentication System Architecture

Kerberos Design Philosophy (MIT Project Athena)

Role: Industry standard Single Sign-On (SSO) authentication system based on symmetric KDCs.

Core Entities:

- Authentication Server (AS),
- Ticket Granting Server (TGS),
- Resource Servers,
- Clients.

Kerberos 2-Step Ticket Handshake

Step 1 (AS Exchange): Client authenticates with password to AS → Receives Ticket-Granting Ticket (TGT) encrypted with K_{TGS}.

Step 2 (TGS Exchange): Client sends TGT to TGS requesting access to Service Bob → Receives Service Ticket $Ticket_{Bob}$.

Step 3 (Service Exchange): Client presents $Ticket_{Bob}$ to Service Bob to establish secure session.

SLIDE 30

Kerberos Ticket Contents & Security Guarantees

Kerberos Ticket Structure

Ticket Contents:

$$Ticket_{Bob} = E_{K_B}(\text{Session Key } K_{AB}, \text{Client Name, Client IP, Start Time, Expiration Time})$$

Authenticator Structure:

$$Auth = E_{K_{AB}}(\text{Client Name, Timestamp } T_{current})$$

Ticket Properties: Unforgeable by client; reusable until expiration time (e.g., 8-10 hours).

Kerberos Security Advantages

- **No Password Transmission**: User password never crosses the network wire (only used locally to decrypt AS response).
- **Single Sign-On (SSO)**: User types password ONCE per day to get TGT, then accesses all enterprise services seamlessly.

SLIDE 31

Secure Replicated Servers & Threshold Signatures

Securing Replicated Server Groups

Context: Active server replicas subject to Byzantine corruptions (up to c corrupted servers).

Requirement: Client must receive a valid response signed by the group without trusting any single replica.

m-of-n Threshold Signature Schemes

(m, n)-**Threshold Concept**: Secret signing key split into n shadows across n replicas; any m shadows reconstruct signature.

Security Property: c corrupted servers (where $c < m$) CANNOT forge a group signature alone.

Client Verification: Client gathers m valid signature shares to verify response correctness.

SLIDE 32

Transport Layer Security (TLS / SSL) Architecture

TLS Protocol Layering

Position: Operates between Transport Layer (TCP) and Application Layer (HTTP/FTP).

Sub-protocols:

- TLS Handshake Protocol,
- TLS Change Cipher Spec,
- TLS Alert Protocol,
- TLS Record Protocol.

TLS 1.3 Handshake Optimization

- 1-**RTT Handshake**: Establishes session key and authenticates server in a single round-trip time.
- **Forward Secrecy (PFS)**: Uses Ephemeral Diffie-Hellman (ECDHE);
  - Compromising long-term keys cannot decrypt past sessions.
- **Zero-RTT (0-RTT) Resumption**: Returning clients can send encrypted application data in the very first packet.

Content

Module 1: Security Threats, Policies, & Architectural Design Principles.

Module 2: Cryptographic Foundations (Symmetric, Asymmetric, Hash Functions).

Module 3: Secure Channels, Authentication Protocols, & Kerberos Architecture.

Module 4: Access Control, Capabilities, Firewalls, & Mobile Code Security.

Module 5: Academic Assessment, Numerical Problems, & Practical Lab.

SLIDE 33

Access Control Abstraction: Subjects, Objects & Operations

Access Control Model Architecture

- **Subject**: An entity issuing requests to perform operations (User, Process, Service Agent).
- **Object**: A resource requiring protection (File, Database Record, Socket Port, Memory Page).
- **Reference Monitor**: Trusted mediator intercepting EVERY request to enforce access control rules.

The Access Control Matrix (ACM)

Conceptual Matrix:

- Rows = $Subjects(S_1..S_m)$;
- Columns = $Objects(O_1..O_n)$.

Cell Entry $A[S_i, O_j]$: Defines exact permitted access rights (e.g., Read, Write, Execute, Delete).

Implementation Challenge: Full matrix is sparse and too huge to store directly in memory.

SLIDE 34

ACLs vs. Capabilities: Dual Matrix Realizations

Access Control Lists (ACLs) - Column-Wise

Organization: Attach access list directly to each OBJECT (Column of ACM).

File ACL Example: File_A → (Alice: `Read/Write`; Bob: `Read`; Charlie: `None`).

- **Advantage**: Easy to revoke or modify permissions for a specific file.
- **Disadvantage**: Hard to determine all files accessible by a specific user.

Capabilities - Row-Wise

Organization: Attach unforgeable tickets directly to each SUBJECT (Row of ACM).

Capability Token: Subject holds ticket specifying Object ID + Permitted Rights + Cryptographic Checksum.

- **Advantage**: Extremely fast access checks; easy to pass permissions between processes.
- **Disadvantage**: Revoking a capability distributed to thousands of subjects is difficult.

SLIDE 35

Firewalls & Network Boundary Defense

Firewall Architectural Purpose

- **Boundary Defense**: Isolates an internal trusted network from an external untrusted network (Internet).
- **Traffic Inspection**: Filters incoming and outgoing packets based on corporate security rules.

Types of Firewalls

1. **Packet Filtering Router**: Inspects IP/TCP headers (Src IP, Dst IP, Port) at Network Layer.

2. **Stateful Packet Inspection (SPI)**: Tracks active TCP connection state tables.

3. **Application-Level Gateway (Proxy)**: Inspects full application payloads (HTTP/FTP) for malicious commands.

SLIDE 36

Secure Mobile Code & Java Virtual Machine Sandbox

Mobile Code Security Hazards

- **Threat**: Downloading and executing untrusted foreign executable code (Java Applets, JavaScript, WebAssembly).
- **Hazards**: Malicious code corrupting local file systems, reading private files, or launching network attacks.

JVM Sandbox Security Architecture

- **Class Loader**: Separates local trusted classes from untrusted network-downloaded classes.
- **Bytecode Verifier**: Scans downloaded class files before execution to ensure no memory corruption or pointer illegalities.
- **Security Manager**: Intercepts all system calls (file access, network sockets) to enforce sandbox restrictions.

Content

Module 1: Security Threats, Policies, & Architectural Design Principles.

Module 2: Cryptographic Foundations (Symmetric, Asymmetric, Hash Functions).

Module 3: Secure Channels, Authentication Protocols, & Kerberos Architecture.

Module 4: Access Control, Capabilities, Firewalls, & Mobile Code Security.

Module 5: Academic Assessment, Numerical Problems, & Practical Lab.

SLIDE 37

Numerical Review Problems: RSA Cryptography Math

Problem 1: RSA Key Generation & Encryption Math

Given Primes: p = 3, q = 11. Compute Modulus n = p * q = 33.

Compute Totient: phi(n) = (3 - 1) * (11 - 1) = 2 * 10 = 20.

Choose Public Exponent: e = 3 (gcd(3, 20) = 1).

Calculate Private Key d: (d * 3) mod 20 = 1 -> d = 7 (since 7 * 3 = 21 = 1 mod 20).

Encryption Test:
Encrypt Plaintext P = 4 -> C = (4^3) mod 33 = 64 mod 33 = 31.

Decryption Test:
Decrypt C = 31 -> P = (31^7) mod 33 = 4 (Verified!).

Problem 2: Key Scaling Math

In a cluster of N = 100 hosts, symmetric pairwise keys required = 100 * 99 / 2 = 4,950 keys.

Using a central KDC, keys required = 100 keys (98% reduction!).

Numerical Review Problems: Threshold Signatures Math

Problem 1: Threshold Signature Scheme Sizing

Given: Replicated group of N = 7 servers. System must tolerate up to c = 2 corrupted servers.

Question: What is the minimum threshold signature parameter m required?

Formula: m = c + 1 = 2 + 1 = 3 signature shares required.

Evaluation: Any 3 server signature shares can construct a valid group response signature.

Problem 2: Diffie-Hellman Key Exchange Calculation

Given Public Parameters: p = 23, g = 5.

Private Choices: Alice picks a = 6; Bob picks b = 15.

Alice Public: A = (5^6) mod 23 = 15625 mod 23 = 8.

Bob Public: B = (5^15) mod 23 = 19.

Shared Key: Alice computes K = (19^6) mod 23 = 2. Bob computes K = (8^15) mod 23 = 2.

SLIDE 39

Practical Programming Lab: Secure RPC Proxy Simulator

Lab Overview & Assignment Goals

- **Assignment Title**: Secure RPC & KDC Authentication Proxy Simulator.
- **Language**: Python, Java, or Go.
- **Architecture**: Implement 3 socket processes: KDC Daemon, Client App, and Secure Resource Server.

Required Implementation Features

1. **KDC Authentication**: Implement Needham-Schroeder ticket granting sequence.

2. **Secure RPC Channel**: Encrypt RPC payloads using AES-CBC with session key K_{AB}.

.3**Integrity Verification**: Attach HMAC-SHA256 checksums to all RPC messages.

4. **Anti-Replay Protection**: Enforce nonce verification to detect and reject replayed RPC packets.

SLIDE 40

Lab Expected Console Trace Log & Evaluation Rubric

Sample Console Output Trace

```
[CLIENT] Requesting Session Key for Server_Bob from KDC (Nonce R_A = 84920)...

[KDC] Ticket_Bob Generated | Session Key K_AB Created.

[CLIENT] Decrypted Response | Ticket_Bob Forwarded to Server_Bob.

[SERVER] Ticket_Bob Decrypted | Nonce Challenge R_B = 10492 Verified.

[CLIENT] Secure Channel Established with Server_Bob!

[CLIENT] Executing Secure RPC: Write('Balance', 500) | Encrypted Payload + HMAC Sent.

[SERVER] HMAC Verified OK | Decrypted RPC Execution Successful.
```

Grading Rubric

- KDC Handshake & Ticket Generation (35%),
- AES Encryption & HMAC Channel Security (30%),
- Nonce Replay Mitigation (25%),
- Documentation & Code Structure (10%).

Recommended Supplementary Resources

Anderson, R. (2008). *Security Engineering: A Guide to Building Dependable Distributed Systems (2nd ed.)*. John Wiley & Sons. (The definitive textbook on engineering secure distributed systems, covering threat modeling, key management, access control, and real-world failure modes.)

Needham, R. M., & Schroeder, M. D. (1978). "Using Encryption for Authentication in Large Networks of Computers". *Communications of the ACM*, 21(12), 993–999. (The landmark paper that introduced symmetric and public-key authentication protocols, forming the foundation of Kerberos.)

Steiner, J. G., Neuman, C., & Schiller, J. I. (1988). "Kerberos: An Authentication Service for Open Network Systems". *Proceedings of the USENIX Winter Conference*, 191–202. (The original paper describing MIT's Kerberos v4/v5 architecture, Ticket-Granting Services, and single sign-on mechanics.)

Diffie, W., & Hellman, M. (1976). "New Directions in Cryptography". *IEEE Transactions on Information Theory*, 22(6), 644–654. (The revolutionary paper that invented Public-Key Cryptography and the Diffie-Hellman key exchange algorithm.)

Saltzer, J. H., & Schroeder, M. D. (1975). "The Protection of Information in Computer Systems". *Proceedings of the IEEE*, 63(9), 1278–1308. (The foundational paper establishing the 8 timeless design principles for computer security (Least Privilege, Complete Mediation, Fail-Safe Defaults, etc.).